QuantumTransportEOM
Detailed User Manual

v0.2.0

Project Documentation

July 6, 2026

Contents

1 Purpose and Scope

QuantumTransportEOM is a standalone Python package for analytical and numerical workflows in quantum transport theory, with special focus on:

- commutator and anticommutator calculations for fermionic, bosonic, and mixed operator algebras,
- equation-of-motion (EOM) closure analysis for predefined and *arbitrary user-supplied* Hamiltonians (`custom_model` / `CustomModel`), with automatic basis expansion and mean-field (Hartree) or Hubbard-I truncations,
- Keldysh contour algebra: Langreth rules, Dyson equations, symbolic Meir–Wingreen currents, and numeric Keldysh self-energies,
- Rashba-Hubbard tight-binding ring Hamiltonians in real space and reciprocal space,
- retarded, advanced, lesser, and greater Green functions, both symbolic and numeric,
- numeric two-terminal devices (`MatrixDevice` and friends) with structured leads (wide band, spin polarized, ferromagnetic, semi-infinite chain, sampled),
- interacting open Anderson dots (Hubbard-I / Hartree self-energies, self-consistent occupations, finite-bias currents),
- transport observables: transmission, conductance, Landauer and Meir–Wingreen currents, spin-resolved variants, persistent currents, and a Drude finite-difference proxy,
- *acceleration*: batched frequency sweeps (stacked LAPACK/BLAS), multi-core CPU execution via `workers=`, and optional CUDA GPU execution via CuPy (`backend="cupy"`).

This manual describes the full package architecture, installation, API behavior, formula conventions, and practical workflows.

2 Project Layout

The standalone folder is:

```
QuantumTransportEOM/
  pyproject.toml
  LICENSE
  run_all.ps1
  README.md
  src/quantum_transport/
    __init__.py          # public API surface
    algebra.py           # commutators, basis decomposition
    secondquant.py       # fermionic wrappers, Wick + exact CAR ordering
    bosonic.py           # bosonic wrappers and EOM
    models.py            # predefined + custom symbolic models
    highlevel.py         # QuTiP-style API, CustomModel, open Anderson
    eom.py               # EOM closure checks
    hamiltonians.py      # Rashba-Hubbard ring builders
    greens.py            # equilibrium Green functions
    keldysh.py           # numeric Keldysh self-energies and views
    keldysh_symbolic.py# contour algebra, Langreth, Dyson
```

```
    devices.py          # MatrixDevice, leads, transport views
    observables.py      # transmission, currents, conductance
    rings.py            # Aharonov-Bohm ring devices
    mean_field.py       # self-consistent Hartree(-Fock)
    transport.py        # persistent currents, Drude proxy
    numerics.py         # backends, batched kernels, parallel sweeps
  examples/             # 20+ runnable demos incl. keldysh guide
  tests/                # 169 tests incl. physics validation
  docs/
    user_manual.tex
    user_manual.pdf
```

3 Installation and Environment

3.1 Python Requirements

- Python 3.10 or newer.
- Core dependencies (installed automatically): `numpy>=2.0`, `sympy>=1.11`, `matplotlib>=3.7`, `threadpoolctl>=3`.
- Optional extras: `[test]` adds `pytest`; `[gpu]` adds `cupy-cuda12x` for CUDA GPU execution (use `cupy-cuda11x` manually on CUDA 11 hosts).

3.2 Editable Install

From `QuantumTransportEOM/`:

```
python -m pip install -e .
python -m pip install -e ".[test]"    # with pytest
python -m pip install -e ".[gpu]"     # CUDA hosts only
```

3.3 Running the Demo

```
$env:PYTHONPATH=".\src"
python .\examples\demo.py
```

3.4 Running Tests

```
python -m pip install pytest
python -m pytest .\tests\test_quantum_transport.py
```

3.5 Automation Script

Use the provided script:

```
.\run_all.ps1
```

Useful switches:

- `-SkipInstall`
- `-SkipDemo`
- `-SkipTests`
- `-InstallPytest`
- `-BuildManual`

4 Mathematical Conventions

4.1 Basis and Indexing

In the numerical modules, the spinful site basis is ordered as

$$(0,\uparrow),(0,\downarrow),(1,\uparrow),(1,\downarrow),\ldots,(N-1,\uparrow),(N-1,\downarrow).$$

Hence the Hamiltonian dimension is $2N \times 2N$.

4.2 Rashba-Hubbard Ring Hamiltonian

The implemented mean-field real-space Hamiltonian is:

$$H = \sum_{n,\sigma} E'_{n\sigma}\, c^\dagger_{n\sigma} c_{n\sigma} + \sum_n \left(c^\dagger_{n+1}\, \Gamma_{n+1,n}\, c_n + \text{h.c.} \right), \tag{1}$$

$$E'_{n\uparrow} = E_{n\uparrow} + U\langle n_{n\downarrow}\rangle, \quad E'_{n\downarrow} = E_{n\downarrow} + U\langle n_{n\uparrow}\rangle. \tag{2}$$

The nearest-neighbor spin matrix $\Gamma_{n+1,n}$ contains both hopping and Rashba terms with Peierls phase

$$\theta = \frac{2\pi}{N}\frac{\phi}{\phi_0}.$$

4.3 Green Functions

For a quadratic Hamiltonian H:

$$G^r(\omega) = \left[(\omega + i\eta)I - H\right]^{-1}, \tag{3}$$

$$G^a(\omega) = \left[(\omega - i\eta)I - H\right]^{-1}. \tag{4}$$

Equilibrium lesser and greater functions:

$$G^<(\omega) = f(\omega)\left(G^a(\omega) - G^r(\omega)\right), \tag{5}$$

$$G^>(\omega) = \left(f(\omega) - 1\right)\left(G^a(\omega) - G^r(\omega)\right), \tag{6}$$

with Fermi function $f(\omega)$.

4.4 Transport Quantities

The package computes:

- $J_c(\omega)$: current density integrand built from matrix elements of $G^r - G^a$, hopping γ, Rashba λ_R, and flux phase factors.

- $I_c(\phi)$: persistent current by numerical integration over ω:

$$I_c(\phi) = \int J_c(\omega)\, d\omega.$$

- Drude proxy:

$$D \sim \frac{dI}{d\phi} \approx \frac{I(\phi + \delta\phi) - I(\phi - \delta\phi)}{2\delta\phi}.$$

5 Module-by-Module API Reference

5.1 `quantum_transport.algebra`

Purpose: symbolic noncommutative algebra utilities.

Function	Behavior
`commutator(a,b,simplify=True)`	Returns $[a, b] = ab - ba$. Uses SymPy expand and optional simplify.
`anticommutator(a,b,simplify=True)`	Returns $\{a, b\} = ab + ba$.
`decompose_in_basis(expr,basis)`	Returns $(\mathbf{c}, r)$ such that $expr = \sum_j c_j\, basis_j + r$. Useful for EOM projection diagnostics.
`operator_symbols(prefix,size)`	Creates noncommutative SymPy symbols (for example `c0,c1,...`).

5.2 `quantum_transport.eom`

Purpose: EOM closure checks and symbolic retarded Green solver on closed bases.

Object / Function	Behavior
`EOMClosureResult`	Dataclass with fields: `operators`, `eom_matrix`, `residuals`. Property `is_closed` is true if all residuals are zero.
`check_eom_closure(operators,H)`	Builds matrix M from $[O_i, H] = \sum_j M_{ij} O_j + \text{residual}_i$.
`retarded_green_from_eom(M,C,omega)`	Solves $(\omega I - M)G^r = C$. Raises if basis is not closed.
`source_anticommutator_matrix(A,B)`	Builds symbolic $C_{ij} = \{A_i, B_j\}$.

5.3 `quantum_transport.hamiltonians`

Purpose: numerical Hamiltonian builders and real↔k-space transforms.

Function	Behavior
`build_rashba_hubbard_ring_real_space(N,...)`	Builds $2N \times 2N$ Hermitian matrix with on-site MF shift, NN hopping, Rashba coupling, and Peierls phase.
`real_to_k_space(h_real,n_sites,spin_dim=2)`	Applies discrete Fourier transform over site index and returns (H_k, k_values).
`split_k_blocks(h_k,n_sites,spin_dim=2)`	Splits H_k into contiguous $spin\_dim \times spin\_dim$ diagonal blocks (per k index in the transformed ordering).

5.4 `quantum_transport.greens`

Purpose: retarded/advanced and equilibrium Keldysh components.

`Function`	Behavior
`fermi_dirac(ω,mu=0,T=0)`	At $T = 0$: returns 1 below μ, 0 above, and 0.5 at equality. At $T > 0$: returns logistic form.
`retarded_green(h,ω,eta)`	Matrix inverse for scalar or vector ω. Returns shape (dim, dim) or (n_ω, dim, dim).
`advanced_green(h,ω,eta)`	Same shape behavior as retarded, with $-i\eta$.
`spectral_function(G_r,G_a)`	Returns $A = i(G^r - G^a)$.
`lesser_green_equilibrium(...)`	Returns $G^< = f(G^a - G^r)$. Supports scalar/vector ω.
`greater_green_equilibrium(...)`	Returns $G^> = (f-1)(G^a - G^r)$.

5.5 `quantum_transport.mean_field`

Purpose: self-consistent collinear-Hartree loop for on-site Hubbard decoupling.

`Object / Function`	Behavior
`HartreeFockResult`	Dataclass containing final Hamiltonian, eigenpairs, $n_\uparrow$, $n_\downarrow$, occupations, iteration count, and convergence flag.
`hartree_fock_self_consistent(...)`	Iterative density update with linear mixing until max density change $<$ `tol` or `max_iter`.

Important implementation details:

- Zero-temperature filling with optional fractional last occupation for non-integer N_e.
- Density matrix construction:
$$\rho = V\,\mathrm{diag}(occ)\,V^\dagger.$$
- Spin densities sampled from diagonal entries in interleaved basis:
$$n_\uparrow(n) = \rho_{2n,2n}, \quad n_\downarrow(n) = \rho_{2n+1,2n+1}.$$

5.6 `quantum_transport.transport`

Purpose: transport observables from Green functions.

`Function`	Behavior
`current_density_omega(...)`	Computes $J_c(\omega)$ in the manuscript-style structure, combining spin-preserving and Rashba spin-flip terms. Returns real scalar.
`persistent_current(...)`	Builds G^r, G^a over ω grid and integrates $J_c(\omega)$ with trapezoidal rule.
`drude_weight(Iplus,Iminus,dphi)`	Central finite difference $(I_+ - I_-)/(2\,d\phi)$.

6 End-to-End Workflows

6.1 Workflow A: HF + Current + Drude

```
import numpy as np
from quantum_transport import (
    hartree_fock_self_consistent,
    persistent_current,
    drude_weight,
)

n_sites = 8
n_electrons = 8
gamma = 1.0
lambda_r = 0.4
u_hubbard = 0.4
phi = 0.2
dphi = 1e-3

hf = hartree_fock_self_consistent(
    n_sites=n_sites,
    n_electrons=n_electrons,
    gamma=gamma,
    lambda_r=lambda_r,
    phi_over_phi0=phi,
    u_hubbard=u_hubbard,
)

mu = 0.5 * (hf.eigenvalues[n_electrons - 1] + hf.eigenvalues[
   n_electrons])
omega_grid = np.linspace(-6.0, 6.0, 2001)

I0 = persistent_current(
    hamiltonian=hf.hamiltonian,
    omega_grid=omega_grid,
    n_sites=n_sites,
    gamma=gamma,
    lambda_r=lambda_r,
    phi_over_phi0=phi,
    mu=mu,
)

hf_plus = hartree_fock_self_consistent(n_sites, n_electrons, gamma,
   lambda_r, phi + dphi, u_hubbard)
hf_minus = hartree_fock_self_consistent(n_sites, n_electrons, gamma,
   lambda_r, phi - dphi, u_hubbard)

Iplus = persistent_current(hf_plus.hamiltonian, omega_grid, n_sites,
   gamma, lambda_r, phi + dphi, mu)
Iminus = persistent_current(hf_minus.hamiltonian, omega_grid, n_sites,
   gamma, lambda_r, phi - dphi, mu)

D = drude_weight(Iplus, Iminus, dphi)
print(I0, D)
```

6.2 Workflow B: Symbolic EOM Closure

```
import sympy as sp
from quantum_transport import commutator, anticommutator,
    check_eom_closure

c0, c1 = sp.symbols("c0 c1", commutative=False)
H = c0*c1 + c1*c0

print(commutator(c0, H))
print(anticommutator(c0, c1))

closure = check_eom_closure([c0, c1], H)
print("Closed basis?", closure.is_closed)
print("Residuals:", closure.residuals)
```

6.3 Workflow C: Real to Reciprocal Space

```
import numpy as np
from quantum_transport import (
    build_rashba_hubbard_ring_real_space,
    real_to_k_space,
    split_k_blocks
)

H = build_rashba_hubbard_ring_real_space(
    n_sites=8,
    gamma=1.0,
    lambda_r=0.3,
    phi_over_phi0=0.1,
)
Hk, k = real_to_k_space(H, n_sites=8, spin_dim=2)
blocks = split_k_blocks(Hk, n_sites=8, spin_dim=2,
    require_block_diagonal=False)

print(k[:3])
print(blocks[0])
```

7 Using `run_all.ps1`

Basic use:

```
.\run_all.ps1
```

Install package and pytest if missing, run demo and tests, and build manual PDF:

```
.\run_all.ps1 -InstallPytest -BuildManual
```

Run only docs build:

```
.\run_all.ps1 -SkipInstall -SkipDemo -SkipTests -BuildManual
```

8 Testing and Validation

Current tests verify:

- commutator and anticommutator identities,
- Hermiticity of the built ring Hamiltonian,
- spectrum invariance under real-to-k transformation,
- shape consistency of $G^r, G^a, G^<, G^>$,
- execution of HF loop with physically valid returned densities.

Recommended additional tests for research-grade studies:

- convergence sensitivity vs mixing and tolerance,
- finite-size scaling of persistent current with N,
- cross-check between derivative-of-energy and Green-function current,
- benchmark against exact diagonalization for small N,
- regression tests for particle-hole symmetry in even bipartite rings.

9 Troubleshooting

9.1 Import Errors

If Python cannot import `quantum_transport`, run:

```
python -m pip install -e .
```

or set:

```
$env:PYTHONPATH=".\src"
```

9.2 pytest Missing

```
python -m pip install pytest
```

or:

```
.\run_all.ps1 -InstallPytest
```

9.3 No PDF Generated

If `pdflatex` is not installed, install MiKTeX or TeX Live, then run:

```
.\run_all.ps1 -BuildManual -SkipInstall -SkipDemo -SkipTests
```

9.4 HF Not Converging

Try:

- lower `mixing` (for example 0.2),
- higher `max_iter`,
- relaxed `tol` for exploratory scans,
- better initial densities if a specific ordered phase is expected.

10 Custom Models and the Exact CAR Engine

10.1 Building a Model from Any Hamiltonian

`CustomModel` (or the lower-level `custom_model`) accepts an arbitrary second-quantized Hamiltonian built from the constructors `f`/`fd` (fermions) and `b`/`bd` (bosons). Statistics (`"fermion"`, `"boson"`, `"mixed"`), the mode content, and a seed operator basis (one annihilator per mode) are detected automatically:

```
import sympy as sp
from quantum_transport import CustomModel, n

eps, U = sp.symbols("epsilon U", real=True)
dot = CustomModel(eps*(n("up") + n("down")) + U*n("up")*n("down"))
dot.model.eom()                          # exact projected EOM
dot.model.eom(auto_expand_steps=1)  # closes on the atomic hierarchy (4
    x4)
dot.model.eom(truncation="hartree") # mean-field closure on the seed
    basis
```

Unless `check_hermitian=False` is passed, a visibly non-Hermitian Hamiltonian (typically a missing conjugate hopping term) raises a `UserWarning`.

10.2 Exact CAR Normal Ordering

For symbolic mode labels (`"up"`, `"L"`, ...) SymPy's `wicks` machinery is unreliable, so custom fermionic models are processed by an exact canonical anticommutation engine (`normal_order_fermionic`): adjacent pairs are rewritten with

$$c_i c_j^\dagger = \delta_{ij} - c_j^\dagger c_i, \tag{7}$$

identical adjacent operators vanish by nilpotency, and same-type runs are sorted canonically with anticommutation signs so equivalent strings cancel. Because `custom_model` enumerates every ladder index as an independent orthogonal mode, Kronecker deltas between *distinct* labels are then collapsed to zero (`collapse_orthogonal_mode_deltas`). The projected EOM matrix accepts only scalar coefficients; operator strings that do not close on the basis land in the residuals, from which `auto_expand_steps` harvests new composite basis operators.

10.3 Generic Hartree Truncation

`truncation="hartree"` applies a Wick-style mean-field factorization to the cubic strings generated by quartic interactions,

$$ABC \longrightarrow \langle AB\rangle C - \langle AC\rangle B + \langle BC\rangle A, \tag{8}$$

keeping density contractions $\langle c_i^\dagger c_i\rangle$ (symbols `n_<i>_avg` unless numbers are supplied via `truncation_params={"...}`). With `include_fock=True` the off-diagonal coherences $\langle c_i^\dagger c_j\rangle$ are retained as symbols. This works for any fermionic model, not just the Anderson impurity.

10.4 Opening a Quadratic Model into a Device

For non-interacting fermionic Hamiltonians $H = \sum_{ij} h_{ij} c_i^\dagger c_j$, the single-particle matrix h is extracted symbolically (`single_particle_hamiltonian_matrix`) and converted into a numeric `MatrixTransportView`:

```
view = model.open({"0": 0.4}, {"2": 0.4}, parameters={"t": 1.0})
```

Lead arguments accept a scalar (wide-band coupling on every site), a mapping from mode labels to couplings (site-resolved contacts), a full coupling matrix, or a `LeadSelfEnergy`. Interacting Hamiltonians raise a `ValueError` here; use the EOM layer with a truncation instead.

11 Acceleration: Batched Kernels, CPU Parallelism, GPU

11.1 Batched Frequency Sweeps

All `*_values` methods on `MatrixTransportView` and `OpenAndersonTransportView` replace the per-frequency Python loop with stacked linear algebra: the retarded Green function for the whole grid,

$$G^r(\omega_k) = \left[(\omega_k + i\eta)\mathbb{1} - H - \Sigma^r(\omega_k)\right]^{-1}, \qquad k = 1, \dots, n, \tag{9}$$

is computed as a single batched inversion of an (n, d, d) stack, and the Landauer trace $T(\omega) = \mathrm{Re}\,\mathrm{tr}[\Gamma_L G^r \Gamma_R G^a]$ as chained batched matrix products. The scalar per-frequency methods remain available and the test suite enforces bit-level agreement between both paths.

11.2 Memory Blocking and CPU Workers

Grids are processed in blocks whose $(n_{\mathrm{block}}, d, d)$ stacks stay under a memory target (default 64 MiB shared across workers), which caps peak memory for large devices. Passing `workers=N` maps blocks onto N threads; NumPy releases the GIL inside LAPACK, and small-matrix factorizations (dimension ≤ 1024) are pinned to a single BLAS thread via `threadpoolctl` — on many-core machines multithreaded OpenBLAS factorizations of small matrices are otherwise up to 30× slower than single-threaded ones. Typical measured gains for a 120-orbital ring on 400 frequencies: ∼30 s scalar loop → 2.4 s batched → <1 s with `workers=8`.

11.3 GPU Backend and Precision

`backend="cupy"` (or `"gpu"`) runs the same kernels on a CUDA device through CuPy; `backend="auto"` selects CuPy when a usable GPU is present and falls back to NumPy otherwise. Install with `pip install -e ".[gpu]"` on a CUDA 12 host.

Consumer (GeForce) GPUs execute float64 at 1/64 of their float32 rate, so complex128 inversions on such cards are no faster than a single CPU core. Pass `precision="single"` to run the batched inversions in complex64 — $\sim 10^{-5}$ relative accuracy and roughly a $10\times$ speedup on an RTX 3080 (600-orbital device, 400 frequencies: 8.5 s CPU $\to$ 1.1 s GPU). Keep the default double precision for sharp resonances (tiny η) or on datacenter GPUs with full float64 throughput. Whether `workers=` beats the serial batched path is machine-dependent (it hinges on the local BLAS); benchmark once with `examples/demo_parallel_gpu.py`.

The helpers `available_backends()`, `get_backend()`, and `to_numpy()` expose the backend layer directly, and `parallel_map` / `blocked_over_grid` serve custom sweeps:

```
from quantum_transport import available_backends, parallel_map
available_backends()      # {"numpy": True, "cupy": False}
T = view.transmission_values(grid, workers=8)          # CPU threads
T = view.transmission_values(grid, backend="cupy", precision="single")
occ = anderson_view.self_consistent_occupations(grid, workers=4)
```

12 Practical Notes for Research Use

- The equilibrium lesser/greater relations implemented here assume closed-system equilibrium.
- For open systems with leads, use the device layer (`MatrixDevice`, `LeadSelfEnergy`) or `AndersonImpurity.open`, which assemble the contact self-energies and Keldysh equations for you.
- Frequency integrals use `numpy.trapezoid` on user-supplied grids; resolve sharp resonances (width $\sim \Gamma$) with an adequate grid, especially at low temperature.
- For production sweeps use `workers=` and, on CUDA hosts, `backend="cupy"`; `parallel_map` parallelizes external parameter scans.
- `tests/test_physics_validation.py` pins the package to exact analytic results (Lorentzian transmission, spectral sum rule, fluctuation–dissipation, current conservation); run it after modifying physics code.

13 Extension Roadmap

Delivered in v0.2.0: lead self-energies and nonequilibrium transport (device layer, open Anderson dots), arbitrary custom models with an exact CAR engine and generic Hartree truncation, batched/parallel/GPU numerics, and analytic physics-validation tests. Suggested next extensions:

1. Exact diagonalization backend for small interacting clusters.
2. Automatic symmetry checks (particle-hole, time reversal, inversion where applicable).
3. Plotting utilities for spectra, $I(\phi)$, and $D(\lambda_R)$.
4. Higher-order EOM truncations (second Born, GW-like) for custom models.
5. Adaptive frequency grids for sharp resonances at low temperature.

14 License and Citation

If used in academic work, cite the corresponding project repository and include the exact commit hash used to generate numerical results.

A Quick API Summary

Name	Summary
`commutator, anticommutator`	Symbolic operator algebra primitives.
`check_eom_closure`	EOM matrix and residual closure check.
`build_rashba_hubbard_ring_real_space`	Spinful real-space Hamiltonian constructor.
`real_to_k_space, split_k_blocks`	Reciprocal-space transform and block extraction.
`retarded_green, advanced_green`	Frequency-domain matrix Green functions.
`lesser_green_equilibrium, greater_green_equilibrium`	Equilibrium Keldysh components.
`hartree_fock_self_consistent`	Mean-field self-consistent solver.
`current_density_omega, persistent_current`	Transport observables from Green functions.
`drude_weight`	Finite difference Drude proxy from current response.
`CustomModel, custom_model`	Arbitrary Hamiltonians: statistics detection, exact EOM, Hartree, open().
`normal_order_fermionic`	Exact CAR normal ordering (wicks-free).
`single_particle_hamiltonian_matrix`	h_{ij} extraction from quadratic H.
`build_fermionic_hartree_eom_system`	Generic mean-field EOM closure.
`dagger_expression`	Hermitian conjugate of second-quantized expressions.
`MatrixDevice, LeadSelfEnergy`	Numeric devices and structured leads.
`AndersonImpurity.open`	Interacting open dot (Hubbard-I, SCF, currents).
`get_backend, available_backends`	NumPy/CuPy backend resolution.
`parallel_map, blocked_over_grid`	CPU-parallel and memory-blocked sweeps.
`batched_retarded_green, batched_transmission`	Stacked Green-function kernels.